BACKEND INTERNSHIP — TASK BRIEF

Authentication & Protected Routes

Add user accounts, JWT-based login, and lock down routes so only authorized users can access them.

DURATION	STACK	DIFFICULTY
7 Days	**JWT, bcrypt, Middleware**	**Intermediate**

Goal of the Week

This week Slotify learns who its users are. You'll build a full authentication system — registration, login, and JWT-based session management — then protect your slot routes so only logged-in users can create or delete slots. This is one of the most important skills in backend development: knowing not just what is being requested, but who is requesting it.

Tasks

1. **User schema.** Create a Mongoose schema for a `User` with fields: `name`, `email`, and `password`. Email must be unique.
2. **Password hashing.** Install `bcryptjs` and hash the user's password before saving it to the database. Never store plain-text passwords.
3. **POST /auth/register.** Build a registration route that validates input, checks for duplicate emails, hashes the password, and saves the new user.
4. **POST /auth/login.** Build a login route that finds the user by email, compares the hashed password with `bcrypt.compare()`, and returns a signed JWT on success.
5. **JWT setup.** Install `jsonwebtoken`. Store the JWT secret in `.env`. Sign tokens with an expiry of 7 days.
6. **Auth middleware.** Write a `protect` middleware function that reads the JWT from the `Authorization` header, verifies it, and attaches the decoded user to `req.user`.
7. **Protect routes.** Apply the `protect` middleware to `POST /slots` and `DELETE /slots/:id` so only logged-in users can use them.
8. **GET /auth/me.** Build a protected route that returns the currently logged-in user's profile based on the JWT.

Suggested Daily Pace

DAY 1–2

User schema, bcrypt hashing, POST /auth/register route.

DAY 3–4

POST /auth/login, JWT signing and verification, store secret in .env.

DAY 5–6

Auth middleware, protect POST and DELETE slot routes, GET /auth/me.

DAY 7

Test full auth flow in Postman, handle edge cases, commit + push, submit for review.

Resources

bcryptjs	npmjs.com/package/bcryptjs — hashing and comparing passwords
jsonwebtoken	npmjs.com/package/jsonwebtoken — signing and verifying JWTs
JWT Explainer	jwt.io/introduction — understand the token structure
API Testing	In Postman, use the Authorization tab → Bearer Token to send JWTs

Submission Checklist

- [ ] Register and login routes work and return correct responses
- [ ] Passwords are hashed in the database — never stored as plain text
- [ ] Login returns a valid JWT that can be decoded at jwt.io
- [ ] Protected routes return 401 when no token is provided
- [ ] GET /auth/me returns the correct user from the token
- [ ] Code pushed to GitHub with at least 3 meaningful commits, repo link shared by end of Day 7